Task Manager V3 Design Document

Version Name

Version 3: File Saving and Loading

Purpose

Version 3 introduces persistence.

In Version 1 and Version 2, tasks only exist while the program is running.

When the user closes the program, all tasks are lost.

Version 3 solves this problem by saving tasks to a file when the program exits and loading tasks when the program starts.

Goal:

```
Create Tasks
↓
Close Program
↓
Tasks Remain Saved
↓
Open Program Again
↓
Tasks Return
```

New Concepts Learned

Version 3 introduces:

- File Reading
- File Writing
- Data Persistence
- Parsing Data
- Converting Objects to Text
- Converting Text Back to Objects

New Class

FileManager

Purpose:

Responsible for all file-related operations.

Responsibilities:

- Save tasks to file
- Load tasks from file
- Handle missing file situations
- Convert file data into Task objects
- Convert Task objects into file data

FileManager should NOT:

- Display menus
- Handle user input
- Manage task collections
- Complete tasks
- Delete tasks

Updated System Architecture

User ↓ Main ↓ TaskManager ↓ Task

Main ↓ FileManager

Responsibilities:

Main:

- Controls program flow
- Knows when to save
- Knows when to load

TaskManager:

- Manages tasks

Task:

- Stores task data

FileManager:

- Saves and loads tasks

File Format

File Name:

```
tasks.txt
```

Location:

Stored in the project folder.

Format:

```
Task Name,false
Task Name,true
Task Name,false
```

Examples:

```
Study Java,false
Go To Gym,true
Finish Homework,false
```

Rules:

- One task per line
- Comma separates task name and completion status
- Task names cannot contain commas

Invalid:

```
Study Java, Chapter 5,false
```

Reason:

Would break file parsing.

Task Class Changes

Existing Constructor

```
Task(String name)
```

Purpose:

Used when the user creates a new task.

Behavior:

```
completed = false
```

automatically.

New Constructor

```
Task(String name, boolean completed)
```

Purpose:

Used when loading tasks from a file.

Examples:

```
new Task("Study Java", false);
```

```
new Task("Go To Gym", true);
```

TaskManager Changes

New Constructor

```
TaskManager(ArrayList<Task> tasks)
```

Purpose:

Allows TaskManager to start with loaded tasks.

Example:

```
ArrayList<Task> tasks = fileManager.loadTasks();

TaskManager manager =
        new TaskManager(tasks);
```

New Method

```
getTasks()
```

Purpose:

Returns the task collection so FileManager can save it.

Example:

```
ArrayList<Task> tasks =
        manager.getTasks();
```

FileManager Fields

Field

```
private String fileName;
```

Example:

```
new FileManager("tasks.txt");
```

Reason:

FileManager should own file information.

Main should not need to know where data is stored.

FileManager Methods

saveTasks(tasks)

Purpose:

Save all tasks to tasks.txt.

Parameters:

```
ArrayList<Task>
```

Returns:

Nothing.

---

saveTasks() Pseudocode

```
saveTasks(tasks):

    open file

    for each task

        get task name

        get completion status

        write:
            name + "," + completion status
```

```
    close file
```

Example Output:

```
Study Java,false
Go To Gym,true
```

---

loadTasks()

Purpose:

Load tasks from tasks.txt.

Returns:

```
ArrayList<Task>
```

---

loadTasks() Pseudocode

```
loadTasks():

    create empty task list

    if file does not exist

        return empty task list

    open file

    for each line

        split line at comma

        get task name

        get completion status

        create Task
```

```
        add task to list

    close file

    return task list
```

Startup Flow

When the program starts:

```
Main Starts
↓
Create FileManager
↓
loadTasks()
↓
Returns ArrayList<Task>
↓
Create TaskManager(tasks)
↓
Run Program
```

Exit Flow

When the user selects Exit:

```
User Chooses Exit
↓
TaskManager.getTasks()
↓
FileManager.saveTasks(tasks)
↓
Display Success Message
↓
Program Closes
```

Missing File Behavior

Scenario:

```
tasks.txt
```

does not exist.

Expected Behavior:

```
Create empty task list
Continue program
```

Program should NOT:

```
Crash
Return null
Require user setup
```

Save and Load Messages

Show user-friendly messages.

Startup:

```
Loading tasks...
Tasks loaded successfully.
```

Exit:

```
Saving tasks...
Tasks saved successfully.
```

Reason:

Helpful for debugging and verifying functionality while learning.

Testing Checklist

Save Test

Steps:

1. Add several tasks
2. Complete some tasks
3. Exit program

Expected:

tasks.txt contains all tasks and statuses.

Load Test

Steps:

1. Save tasks
2. Close program
3. Run program again

Expected:

All tasks return correctly.

Completion Status Test

Steps:

1. Complete a task
2. Exit
3. Reopen

Expected:

Task remains completed.

Missing File Test

Steps:

1. Delete tasks.txt
2. Run program

Expected:

Program starts normally with empty task list.

Empty Task List Test

Steps:

1. Run program with no saved tasks

Expected:

Program works normally.

Version 3 Complete When

- Tasks save successfully
- Tasks load successfully
- Completion status is preserved
- Missing file does not crash program
- Existing V1 and V2 features still work
- All tests pass
- Changes are committed to GitHub

Summary

Version 3 transforms the task manager from a temporary program into a persistent application.

Key idea:

```
Objects
↓
File
```

```
↓
Objects
```

The user can now close the application and continue where they left off the next time the program is opened.